

01_Introduccion_Python

June 25, 2026

1 Introducción a Python

Materia: 01PAO25-25 - Python 01

Docente: Elvis David Pachacama

Estudiante: Erick Cadena

Fecha: Junio 2026

Repositorio: <https://github.com/edcadena94/machin-Lerning2026>

1.1 Principales características de Python

- Lenguaje de propósito general
- Interpretado, no compilado (más flexible y portable)
- Tipado dinámico
- Fuertemente tipado
- Énfasis en la legibilidad
- Lenguaje de alto nivel
- Gestión automática de memoria
- Multiparadigma: orientado a objetos, procedural y funcional
- Indentación para delimitar bloques de código
- Gran librería con módulos para múltiples tareas
- Multiplataforma

1.2 ¿Por qué Python?

“Python is used in pursuits as diverse as data science, film-making, computer science education, IT management, and much more. There really is no computing field that Python has not touched (except maybe kernel development).”

— Kopec, D. (2019). Classic Computer Science Problems in Python

- Uno de los lenguajes más usados en todo el mundo
- Perfecto para introducción a la programación
- Ecosistema amplio con librerías estables
- Comunidad muy participativa y mucha documentación
- Incrementa la productividad del desarrollador
- Fácil integración con herramientas y otros lenguajes
- Multiplataforma y portable

1.3 ¿Qué puedo hacer con Python?

- Herramientas shell (administración de sistemas)
- Desarrollo de interfaces gráficas (GUIs)
- Comunicación por Internet y redes
- Programación de bases de datos
- Computación numérica (NumPy)
- Aprendizaje automático e inteligencia artificial
- Análisis y tratamiento de datos
- Visualización de datos

1.4 Filosofía de Python - El Zen

```
[1]: import this
```

The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!

1.5 Intérprete de Python y ejecución de scripts

1.5.1 ¿Qué es un intérprete?

- Un programa que ejecuta otros programas
- Una capa de software entre tu código y el hardware
- Debe estar instalado para ejecutar código Python

1.5.2 Implementaciones de Python:

- **CPython**: implementación en C (la más común)
- **Jython**: implementación en Java
- **IronPython**: implementación en .NET

1.5.3 Mi primer programa: Hello World

```
[2]: # Imprimir en pantalla
print('Hello World')
```

Hello World

1.5.4 REPL (Read-Eval-Print-Loop)

Sistema interactivo para comunicarse con el ordenador:

- **Read:** El ordenador lee la entrada
- **Evaluate:** El código es procesado
- **Print:** Los resultados se muestran
- **Loop:** Continúa la conversación

1.6 Comandos especiales de Jupyter (IPython)

- `%run`: ejecuta un script externo
- `%time`: mide tiempo de ejecución de una línea
- `%%time`: mide tiempo de ejecución de una celda completa
- `!`: ejecuta comandos de consola
- `%%bash`: ejecuta comandos en bash

```
[3]: # Ejecutar comando de consola - listar archivos
!dir
```

El volumen de la unidad C es OS

El número de serie del volumen es: 7A3E-3D18

Directorio de C:\Users\erick\Documents\machinLernig2026

```
24/06/2026  23:18    <DIR>          .
07/01/2026  19:05    <DIR>          ..
15/01/2026  10:47                44 .gitignore
20/01/2026  17:40    <DIR>          .ipynb_checkpoints
24/06/2026  23:18          10.433 01_Introduccion_Python.ipynb
16/01/2026  15:43          82.060 02_Python_DataTypes.ipynb
16/01/2026  17:06          53.038 03_Python_Control.ipynb
19/01/2026  18:42          48.874 05_Python_Files.ipynb
20/01/2026  18:24          61.919 06_Python_NumPy.ipynb
18/01/2026  22:52    <DIR>          res
                6 archivos          256.368 bytes
                4 dirs  424.554.741.760 bytes libres
```

1.7 Comparación de rendimiento: NumPy vs Python puro

NumPy es una librería optimizada para operaciones numéricas. Veamos qué tan rápido es comparado con listas normales de Python.

```
[4]: import numpy as np

# Crear array de NumPy con 10 millones de elementos
my_arr = np.arange(10000000)

# Crear lista normal de Python con 10 millones de elementos
my_list = list(range(10000000))

print("Datos creados: 10 millones de elementos")
```

Datos creados: 10 millones de elementos

```
[5]: # Tiempo con NumPy
%time for _ in range(10): my_arr2 = my_arr * 2
```

CPU times: total: 484 ms

Wall time: 502 ms

```
[13]: # Tiempo con lista normal de Python
%time for _ in range(10): my_list2 = [x * 2 for x in my_list]
```

CPU times: total: 9.06 s

Wall time: 9.28 s

1.8 Referencias

- González Duque, R. *Python para todos*. Licencia Creative Commons.
- Boschetti, A. y Massaron, L. (2016). *Python Data Science Essentials*, Second edition. Packt.
- Python Software Foundation. [Built-in Functions — Python Documentation](#)
- [Tutorialspoint - Python Tutorial](#)
- Kenneth Reitz (2018). [Code Style — The Hitchhiker's Guide to Python](#)
- [Introduction to the Python Coding Style](#)

```
[ ]:
```